Ran get exact parameter counts for all 3 models
Checked task Get exact parameter counts for all 3 models

Here is the complete, in-depth technical breakdown of **how** and **where** Preprocessing, Model Architectures/Training, and Final Evaluation are implemented for the first three methods:

1. **Baseline 1: EfficientNet-B3**
2. **Baseline 2: MobileNet (MobileNetV3-Large)**
3. **Method 3: Hybrid QCNN (Quantum CNN)**

---

1. Preprocessing Pipeline

What is Common to All 3 Methods?

All three methods load data through the universal pipeline in `common/dataset.py` and `common/transforms.py`:

- **Image Ingestion**: Images are loaded using PIL and strictly converted to 3-channel RGB:
  ```
  image = Image.open(img_path).convert("RGB")
  ```
- **Data Immutability**: All models read from the split manifests (`data/splits/train.csv` , `val.csv` , `test.csv`). No model ever modifies or moves raw images.
- **Corrupted Image Protection**: If an image file is corrupted or unreadable, a neutral gray tensor placeholder is returned rather than crashing training.
- **Label Mapping**: Binary targets are encoded as float tensors: $\mathrm{RG} = 1.0$, $\mathrm{NRG} = 0.0$.
- **Training Data Augmentations** (applied dynamically in memory during training only):
  - `RandomHorizontalFlip(p=0.5)`
  - `RandomVerticalFlip(p=0.5)`
  - `RandomRotation(degrees=15)`
  - `ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1)`
- **ImageNet Tensor Normalization**:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

- **Validation & Test Transforms**: Strictly deterministic—only resizing and normalization are applied; **zero** random augmentations.

What is Specific to Each Methodology?

Feature	EfficientNet-B3	MobileNetV3-Large	Hybrid QCNN
Input Resolution	300×300	224×224	224×224
Why this resolution?	Native resolution of EfficientNet-B3 compound scaling.	Standard lightweight resolution for mobile convolutions.	Keeps feature dimension manageable before quantum angle projection.
Batch Size	16 (fits GPU VRAM with 300×300)	32 (fast mobile throughput)	16 (optimal for quantum simulation steps)
Retinal CLAHE (Ablation)	Optional (`--clahe`)	Optional (`--clahe`)	Optional (`--clahe`)

Code Implementation (`common/transforms.py`):

```
def get_transforms(image_size: int = 224, is_train: bool = True,
use_augmentation: bool = True, use_clahe: bool = False):
    transform_list = []
    if use_clahe:
        transform_list.append(CLAHETransform(clip_limit=2.0))
    transform_list.append(T.Resize((image_size, image_size),
interpolation=T.InterpolationMode.BILINEAR))
    if is_train and use_augmentation:
        transform_list.extend([
            T.RandomHorizontalFlip(p=0.5),
            T.RandomVerticalFlip(p=0.5),
            T.RandomRotation(degrees=15),
            T.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1)
        ])
    transform_list.extend([
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    return T.Compose(transform_list)
```

---

2. Finer Details of Training & Code

Summary Architecture & Parameter Comparison

Metric / Parameter	Baseline 1: EfficientNet-B3	Baseline 2: MobileNetV3-I
Primary Code File	`methods/baseline_efficientnet_b3/model.py`	`methods/baseline_mobile`
Training Entrypoint	`methods/baseline_efficientnet_b3/train.py`	`methods/baseline_mobile`
Total Parameters	$11,090,437$ (~11.09)...	$3,095,345$ (~3.10)...
Trainable Parameters	$11,090,437$	$3,095,345$
Feature Dimension	$1,536$ dims	960 dims
Optimizer	`AdamW` ($lr = 10^4$)	`AdamW` ($lr = 3 \times 10^4$)
Weight Decay	0.0001	0.0001
Loss Function	Focal Loss ($\alpha = 0.33$, $\gamma = 2.0$)	Focal Loss ($\alpha = 0.33$, $\gamma = 2.0$)
GPU Mixed Precision	PyTorch AMP (`autocast`)	PyTorch AMP (`autocast`)
LR Scheduler	`ReduceLROnPlateau(factor=0.5, patience=2)`	`ReduceLROnPlateau(factc patience=2)`
Early Stopping	`patience=6` , on Val ROC-AUC	`patience=6` , on Val ROC-A

Detailed Method 1: EfficientNet-B3

- **Backbone**: 26 Mobile Inverted Bottleneck Conv (MBConv) blocks scaled across width, depth, and resolution.
- **Classifier Head**:

```
# In methods/baseline_efficientnet_b3/model.py
self.backbone.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(1536, 256),
```

```
        nn.BatchNorm1d(256),
        nn.GELU(),
        nn.Dropout(p=0.15),
        nn.Linear(256, 1)  # Binary logit
    )
```

- **Loss Function**: Focal Loss in `common/losses.py`:

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t), \quad \alpha = 0.33, \quad \gamma = 2.0$$

Detailed Method 2: MobileNetV3-Large

- **Backbone**: 15 Inverted Residual blocks with depthwise separable convolutions, squeeze-and-excitation (SE) attention, and Hardswish non-linearities.
- **Classifier Head**:

```
# In methods/baseline_mobilenet/model.py
self.backbone.classifier = nn.Sequential(
    nn.Dropout(p=0.2),
    nn.Linear(960, 128),
    nn.BatchNorm1d(128),
    nn.Hardswish(),
    nn.Dropout(p=0.1),
    nn.Linear(128, 1)  # Binary logit
)
```

Detailed Method 3: Hybrid Classical-Quantum CNN (QCNN)

The QCNN architecture consists of 4 distinct sequential modules:

```
[Input 224x224x3]
       │
       ▼
[Module 1: Frozen MobileNetV3-Small Backbone] (927,008 params, frozen)
       │  Extracts 576-dimensional classical visual feature vector
       ▼
[Module 2: Classical Angle Encoder] (4,616 trainable params)
       │  Dropout(0.2) -> Linear(576, 8) -> Tanh() * pi
       ▼
[Module 3: PennyLane Parameterized Quantum Circuit] (48 trainable quantum
params)
```

```
        |  • 8 Qubits (Wires 0 to 7)
        |  • AngleEmbedding(rotation='Y')
        |  • StronglyEntanglingLayers(2 layers, weights: 2 x 8 x 3 = 48
params)
        |  • Multi-wire PauliZ expectation readout: <Z_i> in [-1, 1]^8
        ▼
[Module 4: Classical Readout Head] (161 trainable params)
        |  Linear(8, 16) -> ReLU() -> Linear(16, 1)
        ▼
 [Scalar Referral Logit z]
```

Exact Parameter Breakdown of the 4,825 Trainable Parameters:

1. `angle_encoder`: 57×8 $8 = ,6$
2. `quantum_layer` (`StronglyEntanglingLayers`): $2 \times 8 \times 3 = 8$
3. `readout_head`: 8×16 $16, 16 \times 1$ $1, = 6$
4. $= 4616$ 48 $161 = ,8$

Dual Learning Rate Optimizer (`methods/qcnn/train.py`):

To prevent variational quantum barren plateaus from stalling, quantum parameters receive $2\times$ **the learning rate** of classical layers:

```
optimizer = optim.AdamW([
    {"params": classical_params, "lr": 0.0005},
    {"params": quantum_params,   "lr": 0.0010}  # 2x learning rate
], weight_decay=0.0001)
```

---

3. Final Evaluation Metrics

Where It Is Done:

- **Metric Logic & Math**: `evaluation/metrics.py`
- **Single Evaluation Script**: `evaluation/evaluate.py`
- **Official Evaluation Protocol**: `evaluation/evaluation_policy.md`

What All Metrics Are Computed:

1. Primary Clinical Metric: SE@95SP (Sensitivity at 95% Specificity)

- **Why it matters**: The gold-standard primary metric of the international AIROGS Challenge. In population screening, false alarms must be capped at 5% (Specificity $\geq 95\%$).
- **Mathematical Formula**:

$$\text{pAUC}_{95} = \int_{0}^{0.05} \text{TPR}(\text{FPR}) \, d\text{FPR}$$

- **Code Implementation** (`evaluation/metrics.py`):

```python
def compute_se_at_95sp(fpr, tpr):
    valid_indices = np.where(fpr <= 0.05)[0]
    if len(valid_indices) == 0:
        return 0.0
    return float(np.max(tpr[valid_indices]))
```

2. Discriminative Power: ROC-AUC and PR-AUC

- **ROC-AUC**: Threshold-independent probability that a randomly chosen RG case receives a higher predicted risk score than an NRG case.
- **PR-AUC**: Area under the Precision-Recall curve. Crucial for assessing performance under the imbalanced 32.7% positive prevalence.

3. Optimal Operating Threshold Selection () via Youden's J Statistic

Rather than arbitrarily fixing the classification threshold at 0.5, Youden's J Index finds the optimal operating point:

$$J = \max_{t} (\text{TPR}(t) - \text{FPR}(t))$$

$$t^* = \arg\max_{t} J$$

4. Operational Classification Metrics (computed at optimal t^* and baseline $t = 0.5$):

- **Sensitivity (Recall / True Positive Rate)**: TP (fraction of real glaucoma cases successfully referred).
- **Specificity (True Negative Rate)**: TN (fraction of healthy eyes cleared without alarm).
- **Precision (Positive Predictive Value)**: P (clinician referral confidence).
- **F1-Score**: Harmonic mean of Precision and Recall.
- **Balanced Accuracy**: B.
- **Overall Accuracy**: A

5. Probability Calibration: Brier Score

Measures the mean squared calibration error between continuous model confidence and true outcome:

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$$

(Lower is better; 0.0539 for EfficientNet vs 0.1610 for QCNN).

6. Computational Efficiency & Latency

- **Inference Latency**: Precise hardware inference time per image in milliseconds (ms)
- **Throughput**: Processed frames per second (FPS)

What Files and Artifacts Are Produced?

When `evaluation/evaluate.py` runs on any model, it saves into `outputs/evaluations/<method>/`:

1. **`evaluation_report.txt`**: Formatted clinical report detailing all thresholds, confusion matrix counts, and clinical risk numbers.
2. **`metrics_summary.json`**: Machine-readable JSON dictionary of all scores.
3. **`test_predictions.csv`**: Complete image-by-image manifest containing `image_id`, `true_label`, `predicted_prob`, and `predicted_class`.
4. **`roc_curve.png`**: High-resolution ROC curve with the 95% specificity vertical cutoff highlighted.
5. **`pr_curve.png`**: Precision-Recall curve showing PR-AUC.
6. **`confusion_matrix.png`**: Heatmap displaying True Negatives, False Positives (unnecessary alarms), False Negatives (missed glaucoma), and True Positives.
7. **`outputs/evaluations/benchmark_comparison.md`**: Combined side-by-side benchmark table comparing all models simultaneously.